CHAPTER 5 — SOFTWARE DESIGN

Multiple-Choice Questions (MCQ)

Section 1: Official Course-Slide MCQs (12 Items)

Q5.1. The most desirable form of coupling is:

(a) Control Coupling · (b) Data Coupling · (c) Common Coupling · (d) Content Coupling

> **(b) Data Coupling**
>
> **Modules communicate by passing data items as parameters.**
>
> *Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 208; Companion Slides 16, 94.*

Q5.2. The worst type of coupling is:

(a) Content coupling · (b) Common coupling · (c) External coupling · (d) Data coupling

> **(a) Content coupling**
>
> **Content coupling is the tightest, most pathological, and worst form of coupling because one module directly modifies or branches into the internal procedural code, execution state, or local data of another module.** This destroys modular independence and prevents any isolated maintenance or unit testing.
>
> *Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 210; Companion Slide 24, 94.*

Q5.3. The most desirable form of cohesion is:

(a) Logical cohesion · (b) Procedural cohesion · (c) Functional cohesion · (d) Temporal cohesion

> **(c) Functional cohesion**
>
> **Functional cohesion represents the highest, strongest, and most desirable level of cohesion because all elements and instructions within the module cooperate to execute a single, well-defined functional task with no extraneous side effects.** Examples include mathematical functions like compute_square_root() or calculate_sine().
>
> *Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 212; Companion Slide 30, 94.*

Q5.4. The worst type of cohesion is:

(a) Temporal cohesion · (b) Coincidental cohesion · (c) Logical cohesion · (d) Sequential cohesion

> **(b) Coincidental cohesion**
>
> **Coincidental cohesion is the weakest, least desirable, and lowest form of cohesion because instructions are bundled into a module arbitrarily with no meaningful functional, sequential, procedural, or temporal relationship.** It typically arises when random utility routines are lumped together without structural cohesion.
>
> *Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 211; Companion Slide 26, 94.*

Q5.5. Which one is not a strategy for design?

(a) Bottom up design · (b) Top down design · (c) Embedded design · (d) Hybrid design

> **(c) Embedded design**
>
> **Software design methodology establishes three primary classical architectural strategies: Top-Down design, Bottom-Up design, and Hybrid (Mixed) design.** Embedded design is not an architectural decomposition strategy; in software engineering, "embedded" refers to an operational application category, hardware constraint, or a COCOMO development mode.
>
> *Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 213–215; Companion Slides 32–35, 94.*

Q5.6. Temporal cohesion means:

(a) Cohesion between temporary variables · (b) Cohesion between local variable · (c) Cohesion with respect to time · (d) Coincidental cohesion

> **(c) Cohesion with respect to time**
>
> **Temporal cohesion groups operations that are executed within the same window of time during program lifecycle execution, regardless of whether they share data or functional logic.** A canonical example is an application startup routine that initializes variables, opens log files, configures database handles, and draws UI frames simultaneously.
>
> *Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 211; Companion Slide 28, 95.*

Q5.7. Functional cohesion means:

(a) Operations are part of single functional task and are placed in same procedures · (b) Operations are part of single functional task and are placed in multiple procedures · (c) Operations are part of multiple tasks · (d) None of the above

> **(a) Operations are part of single functional task and are placed in same procedures**
>
> **Functional cohesion occurs when all operations contained within a single procedure or module contribute directly and exclusively to executing one focused functional objective.** Breaking a single functional task across multiple arbitrary modules impairs cohesion and introduces undesirable stamp or control coupling.
>
> *Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 212; Companion Slide 30, 95.*

Q5.8. When two modules refer to the same global data area, they are related as:

(a) External coupled · (b) Data coupled · (c) Content coupled · (d) Common coupled

> **(d) Common coupled**
>
> **Common coupling occurs whenever two or more independent modules read, write, or bind to the same shared global data area (such as Fortran COMMON blocks, C global variables, or shared public memory pools).** If one module mutates the shared global format, all other modules referencing that pool risk runtime corruption.
>
> *Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 209; Companion Slide 22, 95.*

Q5.9. The module in which instructions are related through flow of control is:

(a) Temporal cohesion · (b) Logical cohesion · (c) Procedural cohesion · (d) Functional cohesion

> **(c) Procedural cohesion**
>
> **Procedural cohesion exists when elements inside a module are grouped together because they must be executed in a specific, predetermined sequence dictated by program control flow (such as step 1 followed by step 2 followed by step 3).** Unlike sequential cohesion, operations in procedural cohesion do not necessarily pass output data from one step to the next as input.
>
> *Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 211; Companion Slide 27, 95.*

Q5.10. The relationship of data elements in a module is called:

(a) Coupling · (b) Cohesion · (c) Modularity · (d) None of the above

> **(b) Cohesion**
>
> **Cohesion measures the internal binding, intra-module strength, and semantic closeness of elements, statements, and data structures encapsulated within a single module.** Coupling, by contrast, measures inter-module relationship and external interdependence across different modules.
>
> *Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 210; Companion Slide 25, 96.*

Q5.11. A system that does not interact with external environment is called:

(a) Closed system · (b) Logical system · (c) Open system · (d) Hierarchal system

> **(a) Closed system**
>
> **In general systems theory applied to software architecture, a closed system is completely self-contained and operates in total isolation without accepting inputs from, or delivering outputs to, its external operating environment.** Real-world software systems are virtually always open systems because they interact continuously with users, sensors, and external networks.
>
> *Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 204; Companion Slide 6, 96.*

Q5.12. The extent to which different modules are dependent upon each other is called:

(a) Coupling · (b) Cohesion · (c) Modularity · (d) Stability

> **(a) Coupling**
>
> **Coupling is the qualitative and structural metric that gauges the degree of interdependence, interconnection, and mutual reliance between distinct software modules.** Good software engineering practice strives to minimize inter-module coupling while maximizing intra-module cohesion.
>
> *Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 207; Companion Slide 15, 96.*

Section 2: Curriculum Gap-Closing Synthesis MCQs (13 Items)

Q5.13. What is the fundamental operational distinction between Conceptual Design and Technical Design?

(a) Conceptual design specifies data structures and algorithms, while technical design gathers user stories.
(b) Conceptual design is written for the customer detailing *what* the system will do, while technical design is written for developers detailing *how* the system will be built.
(c) Conceptual design is performed after coding, while technical design precedes requirements elicitation.
(d) Conceptual design produces object code, while technical design produces test suites.

> **(b) Conceptual design is written for the customer detailing *what* the system will do, while technical design is written for developers detailing *how* the system will be built.**
>
> **Conceptual design establishes an external, user-oriented model that describes system capabilities and behavior from the stakeholder's perspective without exposing internal mechanics.** Technical design translates those conceptual abstractions into concrete implementation blueprints for software engineers, specifying internal data structures, module interfaces, algorithm choices, and system architecture.
>
> *Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 204–205; Companion Slides 4–5.*

Q5.14. Under Bertrand Meyer's five criteria for software modularity, which criterion requires that an error or exceptional condition occurring within a module remains confined to that module without crashing the entire system?

(a) Modular Decomposability · (b) Modular Composability · (c) Modular Continuity · (d) Modular Protection

> **(d) Modular Protection**
>
> **Modular Protection means that an abnormal condition occurring in a module should have its effects constrained within that module.**
>
> *Source: Bertrand Meyer,* Object-Oriented Software Construction*; Pressman (5th ed.), §13.4.3, printed p. 345.*

> **ملاحظة أكاديمية هامة (Academic Note)**
>
> **ملاحظة الطالب والفخ الامتحاني:**
>
> - معيار الحماية النمطية (Modular Protection) عند Meyer ينص على حصر الأخطاء والاستثناءات داخل الوحدة ومنع انهيار النظام ككل. لا تخلطه مع Modular Continuity التي تعني حصر التغييرات في الكود عند تعديل المواصفات.

Q5.15. When two modules communicate by passing an entire composite data structure (such as a full Employee_Record struct), but the receiving module only uses a single field (Zip_Code), what type of coupling is created?

(a) Data Coupling · (b) Control Coupling · (c) Stamp Coupling · (d) External Coupling

> **(c) Stamp Coupling**
>
> **Stamp coupling occurs when an entire composite data structure or record layout is passed between modules when only a subset of its fields is required by the callee.** This exposes unnecessary fields to accidental manipulation and tightly couples both modules to the structural layout of the entire composite record.
>
> *Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 208; Companion Slide 17.*

Q5.16. In which form of cohesion do module elements operate on the exact same input data set to produce different, independent output values?

(a) Communicational Cohesion · (b) Sequential Cohesion · (c) Logical Cohesion · (d) Temporal Cohesion

> **(a) Communicational Cohesion**
>
> **Communicational cohesion occurs when all operations within a module manipulate or reference the identical input data structure or database record, but each operation produces distinct, separate results.** For example, a module that accepts a student record and concurrently computes GPA, evaluates credit standing, and formats a mailing address possesses communicational cohesion.
>
> *Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 211; Companion Slide 29.*

Q5.17. Which design strategy begins by assembling and testing existing low-level reusable components and utility libraries, gradually constructing higher-level abstractions toward the root of the hierarchy?

(a) Top-Down Design · (b) Bottom-Up Design · (c) Stepwise Refinement · (d) Transform Centered Design

> **(b) Bottom-Up Design**
>
> **Bottom-up design starts with lower-level components and combines them into higher-level components.** It can be useful when reusable components already exist.
>
> *Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 213–214; Companion Slide 33.*

Q5.18. How does a Structure Chart fundamentally differ in semantic meaning from a standard Flowchart?

(a) A structure chart depicts runtime execution paths of algorithms, while a flowchart models class inheritance.
(b) A structure chart shows static calling hierarchy, module partitioning, and data/control couples, whereas a flowchart shows sequential instruction order and runtime control logic.
(c) Structure charts are used exclusively for object-oriented systems, while flowcharts are used for relational databases.
(d) There is no semantic difference; a structure chart is merely a modern graphic synonym for a flowchart.

(b) A structure chart shows static calling hierarchy, module partitioning, and data/control couples, whereas a flowchart shows sequential instruction order and runtime control logic.

A structure chart shows module hierarchy and the data or control information passed between modules. A flowchart shows the steps and branches of a procedure.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 216–218; Companion Slides 41–43.

Q5.19. In functional procedure layers, what is the primary role of the Structural Layer?

(a) Describing step-by-step algorithms using pseudocode.
(b) Specifying machine-executable byte code and hardware register assignments.
(c) Defining module boundaries, calling relationships, and inter-module interface contracts.
(d) Generating user-interface wireframes and customer surveys.

(c) Defining module boundaries, calling relationships, and inter-module interface contracts.

The structural layer of functional design notations models the macro-architecture of the software, establishing module decomposition, hierarchy, and inter-module coupling boundaries. It is complemented by the procedural layer, which articulates the internal algorithmic logic of each discrete module.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 220–221; Companion Slides 47–49.

Q5.20. Under IEEE Std 1016 (Recommended Practice for Software Design Descriptions), what is the core purpose of a Software Design Description (SDD)?

(a) To document developer employment contracts and billable hours.
(b) To serve as an authoritative technical blueprint communicating architecture, module interfaces, and design decisions among stakeholders.
(c) To replace compiler syntax checking with manual inspections.
(d) To act as a commercial marketing brochure for end users.

> **(b) To serve as an authoritative technical blueprint communicating architecture, module interfaces, and design decisions among stakeholders.**
>
> **IEEE Std 1016 specifies that the primary purpose of an SDD is to provide a precise, rigorous reference document that translates customer requirements into architectural blueprints, guiding coders during implementation, testers during verification, and maintainers during system evolution.**
>
> *Source: IEEE Std 1016-1998; K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 221–222; Companion Slides 50–52.*

Q5.21. Which statement best distinguishes an Abstract Data Type (ADT) from a class as presented in the supporting sources?

(a) An ADT specifies operations while hiding representation; a class groups objects with common structure and behavior. · (b) ADTs always use static binding, while classes always use dynamic binding. · (c) An ADT is a machine-level executable, while a class is only a user requirement. · (d) Classes cannot represent data abstractions.

> **(a) An ADT specifies operations while hiding representation; a class groups objects with common structure and behavior.**
>
> **An ADT is characterized by its operations rather than its internal representation. A class is a grouping of objects that share characteristics and behavior and can serve as their blueprint.** These descriptions are related but do not establish a universal class-versus-ADT capability checklist.
>
> *Source: Sommerville (9th ed.), §13.4, PDF p. 356; K.K. Aggarwal & Yogesh Singh (3rd ed.), printed p. 227.*

> 📝 **ملاحظة أكاديمية هامة (Academic Note)**
>
> **ملاحظة الطالب والفخ الامتحاني:** - لا تحفظ ادعاءات أن ADT لا يملك وراثة أو تعدد أشكال؛ هذا الفرق المحدد هو الذي تسنده الصفحات المذكورة.

Q5.22. In architectural structural design, what do the metrics "Fan-in" and "Fan-out" represent for a given module?

(a) Fan-in is the number of lines of code; Fan-out is the number of compiler errors.
(b) Fan-in is the number of modules that call the given module; Fan-out is the number of modules directly called by the given module.
(c) Fan-in is the number of input parameters; Fan-out is the return data type size.
(d) Fan-in is processor clock speed; Fan-out is network bandwidth consumption.

> **(b) Fan-in is the number of modules that call the given module; Fan-out is the number of modules directly called by the given module.**
>
> **High fan-in may indicate reuse. High fan-out may indicate that a module has many subordinate calls; inspect the design before deciding whether it needs restructuring.**
>
> *Source: Pressman (5th ed.), §13.4.5; Rajib Mall (4th ed.), §5.4, pp. 300–301.*

Q5.23. What fundamental architectural principle organizes modules around design decisions that each hides from other modules?

(a) Stepwise Refinement · (b) Information Hiding · (c) Temporal Coupling · (d) Data Pipelining

> **(b) Information Hiding**
>
> **Information hiding organizes modules around design decisions that are hidden from other modules.** A change to a hidden decision can then be localized when the interface remains suitable.
>
> *Source: David Parnas (1972); Pressman (5th ed.), §13.4.4, printed p. 352.*

Q5.24. During DFD-to-Structure-Chart mapping, what is the primary difference between Transform Analysis and Transaction Analysis?

(a) Transform analysis is used for real-time operating systems, while transaction analysis is used for compilers.
(b) Transform analysis identifies a central transform between afferent (input) and efferent (output) streams, while transaction analysis branches control from a central transaction center based on an action tag.
(c) Transform analysis converts classes into objects, while transaction analysis converts tables into SQL views.
(d) There is no structural difference; both algorithms generate identical single-branch structure charts.

> **(b) Transform analysis identifies a central transform between afferent (input) and efferent (output) streams, while transaction analysis branches control from a central transaction center based on an action tag.**
>
> **Transform analysis isolates the incoming data paths (afferent branch), the primary computational kernel (central transform), and outgoing result paths (efferent branch).** Transaction analysis applies when an input data item (transaction) contains an identifying type or opcode that triggers execution of one of several mutually exclusive operational action paths.
>
> *Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 218–220; Rajib Mall (4th ed.), §6.4.1.*

Q5.25. Can a non-trivial software system ever be completely decoupled, such that the degree of coupling between all modules is zero?

(a) Yes, complete decoupling is the mandatory standard requirement of object-oriented architectures.
(b) Yes, complete decoupling is achieved by eliminating all functions and writing all code in a single file.
(c) No, because modules must communicate data and transfer control to execute meaningful system tasks; a completely decoupled system would consist of isolated components unable to cooperate.
(d) No, because compiler standards mandate at least 15 global variables in every software product.

> **(c) No, because modules must communicate data and transfer control to execute meaningful system tasks; a completely decoupled system would consist of isolated components unable to cooperate.**
>
> **Complete decoupling ($Coupling = 0$) implies absolute independence with zero parameter passing, zero shared memory, zero message passing, and zero invocation.** Because software applications exist to solve complex problems through coordinated component collaboration, coupling cannot be entirely eliminated; software engineering aims to achieve *loose coupling* rather than *zero coupling*.
>
> *Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), Exercise 5.24, printed p. 249; Companion Slide 15.*

Topic-to-Question Map

Topic Area	Questions Covering Item	Primary Course Sources	Key Concepts Covered
1. Software Design Fundamentals	Q5.11, Q5.13	K.K. Aggarwal pp. 203–206; Slides 2–9	Conceptual vs technical design, system boundary, closed systems, transition from requirements to architecture.
2. Modularity & Meyer's Criteria	Q5.14	Pressman §13.4.3; K.K. Aggarwal pp. 206–207	Modular decomposability, composability, understandability, continuity, protection.
3. Module Coupling Taxonomy	Q5.1, Q5.2, Q5.8, Q5.12, Q5.15, Q5.25	K.K. Aggarwal pp. 207–210; Slides 15–24	Data, stamp, control, external, common, content coupling; limits of complete decoupling.
4. Module Cohesion Taxonomy	Q5.3, Q5.4, Q5.6, Q5.7, Q5.9, Q5.10, Q5.16	K.K. Aggarwal pp. 210–213; Slides 25–30	Coincidental, logical, temporal, procedural, communicational, sequential, functional cohesion.
5. Design Strategies & Notations	Q5.5, Q5.17, Q5.18, Q5.19	K.K. Aggarwal pp. 213–221; Slides 32–49	Top-down, bottom-up, hybrid design; flowchart vs structure chart; functional procedure layers.
6. IEEE SDD (Std 1016-1998)	Q5.20	K.K. Aggarwal pp. 221–225; Slides 50–57	Scope, definitions, purpose, design entities, attributes, view representations.
7. Object-Oriented Design (OOD)	Q5.21, Q5.22	K.K. Aggarwal pp. 225–249; Slides 58–93	Objects, classes, ADT vs class, inheritance, polymorphism, encapsulation, reusable design guidelines.
8. Architectural & Structural Metrics	Q5.23, Q5.24	Pressman §13.4.5; Mall §5.4, §6.4.1	Fan-in/fan-out, information hiding, DFD transform analysis vs transaction analysis.
